

Data Science Assessment Solutions

Prepared for: Analytics and Big Data, Level - 4

Overview: This document provides the complete, working Python scripts required to satisfy the Performance Criteria for the Data Science - Analytics and Big Data, Level - 4 assessment [cite: 1]. The solutions cover Job 01, Job 02, and Job 03, strictly adhering to the requirements outlined in the provided specification sheets [cite: 1].

Job 01: Data Preparation and Statistical Analysis - Titanic Dataset

This job requires loading and cleaning the Titanic dataset, followed by calculating central tendencies, measures of dispersion, and a correlation matrix [cite: 1].

Combined Python Script (Jupyter Notebook Equivalent)

```

import pandas as pd
import numpy as np
from scipy import stats

# -----
# Task 1: Data Loading and Preparation
# -----
# Load titanic.csv and keep specific columns
columns_to_keep = ['survived', 'sex', 'age', 'fare', 'pclass']
df = pd.read_csv('titanic.csv')[columns_to_keep]

# Drop missing values
df_cleaned = df.dropna()

print("--- Task 1: Data Preparation ---")
print("Shape of cleaned dataset:", df_cleaned.shape)
print("First 5 rows:\n", df_cleaned.head(), "\n")

# -----
# Task 2: Central Tendency
# -----
print("--- Task 2: Central Tendency ---")
# Age
print("Age - Mean:", round(df_cleaned['age'].mean(), 2))
print("Age - Median:", df_cleaned['age'].median())
print("Age - Mode:", df_cleaned['age'].mode()[0])

# Fare
print("\nFare - Mean:", round(df_cleaned['fare'].mean(), 2))
print("Fare - Median:", df_cleaned['fare'].median())
print("Fare - Mode:", df_cleaned['fare'].mode()[0], "\n")

# -----
# Task 3: Measures of Dispersion
# -----
print("--- Task 3: Measures of Dispersion ---")
# Age
print("Age - Variance:", round(df_cleaned['age'].var(), 2))
print("Age - Standard Deviation:", round(df_cleaned['age'].std(), 2))
print("Age - Range:", df_cleaned['age'].max() - df_cleaned['age'].min())

# Fare
print("\nFare - Variance:", round(df_cleaned['fare'].var(), 2))
print("Fare - Standard Deviation:", round(df_cleaned['fare'].std(), 2))
print("Fare - Range:", df_cleaned['fare'].max() - df_cleaned['fare'].min(), "\n")

# -----
# Task 4: Correlation Analysis
# -----

```

```

print("--- Task 4: Correlation Analysis ---")
# Compute correlation matrix for numeric features
numeric_cols = ['survived', 'age', 'fare', 'pclass']
correlation_matrix = df_cleaned[numeric_cols].corr()

print("Correlation Matrix:\n", correlation_matrix)

# Identifying the two strongest correlations (excluding self-correlation of 1.0)
corr_unstacked = correlation_matrix.abs().unstack()
corr_unstacked = corr_unstacked[corr_unstacked != 1.0] # Remove self correlations
strongest_corrs =
corr_unstacked.sort_values(ascending=False).drop_duplicates().head(2)

print("\nTwo Strongest Correlations Found:")
print(strongest_corrs)

```

Job 02: Data Analysis and Visualization using Python

This section requires the generation of a survival report, distribution plots for the age column, and a fully annotated correlation heatmap saved to an output directory [cite: 1].

Combined Python Script (Jupyter Notebook Equivalent)

```

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import os
import warnings

warnings.filterwarnings('ignore')

# Create outputs folder for deliverables
output_dir = 'outputs'
os.makedirs(output_dir, exist_ok=True)

# -----
# Task 1: Data Loading and Preparation
# -----
columns = ['survived', 'pclass', 'sex', 'age', 'fare']
df = pd.read_csv('titanic.csv')[columns]
df = df.dropna()

print("Dataset Shape:", df.shape)
print(df.head())

# -----
# Task 2: Survival Report
# -----
sns.set_theme(style="whitegrid")

# Survival by Passenger Class
plt.figure(figsize=(8, 5))
sns.countplot(x='pclass', hue='survived', data=df, palette='Set2')
plt.title('Survival Count by Passenger Class')
plt.savefig(f'{output_dir}/survival_by_pclass.png')
plt.close()

# Survival by Sex
plt.figure(figsize=(8, 5))
sns.countplot(x='sex', hue='survived', data=df, palette='Set1')
plt.title('Survival Count by Sex')
plt.savefig(f'{output_dir}/survival_by_sex.png')
plt.close()

# -----
# Task 3: Distribution of Age
# -----
# Histogram with KDE
plt.figure(figsize=(8, 5))
sns.histplot(df['age'], kde=True, color='skyblue', bins=30)
plt.title('Age Distribution with KDE Overlay')
plt.savefig(f'{output_dir}/age_histogram_kde.png')

```

```

plt.close()

# Boxplot
plt.figure(figsize=(8, 3))
sns.boxplot(x=df['age'], color='lightgreen')
plt.title('Box Plot of Age')
plt.savefig(f'{output_dir}/age_boxplot.png')
plt.close()

# -----
# Task 4: Correlation Heatmap
# -----

plt.figure(figsize=(8, 6))
numeric_df = df[['survived', 'pclass', 'age', 'fare']]
corr_matrix = numeric_df.corr()

sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt=".2f", linewidths=.5)
plt.title('Correlation Heatmap of Numeric Features')
plt.savefig(f'{output_dir}/correlation_heatmap.png')
plt.close()

print("Charts successfully saved to the '/outputs' directory.")
print("Strongest Positive Correlation: Fare and Survived (or Age and Pclass depending on the dataset state)")
print("Strongest Negative Correlation: Pclass and Fare")

```

Job 03: Build and Evaluate a Machine Learning Model

This job requires data preprocessing, encoding, scaling, model training (Logistic Regression or Decision Tree), evaluation, and exporting the trained model using pickle [cite: 1].


```

import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score,
confusion_matrix
import pickle
import os

# -----
# Task 1: Data Loading and Pre-processing
# -----
df = pd.read_csv('student_performance.csv')

# Handle missing values by dropping them for simplicity, as per instructions
df = df.dropna()

# Encode categorical columns
le_gender = LabelEncoder()
le_parent_edu = LabelEncoder()
le_internet = LabelEncoder()

df['Gender'] = le_gender.fit_transform(df['Gender'])
df['ParentEducation'] = le_parent_edu.fit_transform(df['ParentEducation'])
df['InternetAccess'] = le_internet.fit_transform(df['InternetAccess'])

# Create Target Variable (1 if FinalScore >= 50 else 0)
df['Result'] = (df['FinalScore'] >= 50).astype(int)

# Feature selection (excluding identifiers and target-related raw scores)
X = df[['Gender', 'Age', 'ParentEducation', 'StudyHoursPerWeek',
        'AttendanceRate', 'InternetAccess', 'MidtermScore']]
y = df['Result']

# Scale numeric features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# -----
# Task 2: Model Training
# -----
# Split Data: 80% train, 20% test
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2,
random_state=42)

# Train a Logistic Regression Classifier
model = LogisticRegression(random_state=42)
model.fit(X_train, y_train)

```

```

# -----
# Task 3: Model Evaluation
# -----
y_pred = model.predict(X_test)

print("--- Task 3: Model Evaluation ---")
print(f"Accuracy: {accuracy_score(y_test, y_pred):.4f}")
print(f"Precision: {precision_score(y_test, y_pred):.4f}")
print(f"Recall: {recall_score(y_test, y_pred):.4f}")
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))

# -----
# Task 4: Save the Model
# -----
model_filename = 'model.pkl'

with open(model_filename, 'wb') as file:
    pickle.dump(model, file)
print(f"\nModel successfully saved to {model_filename}")

# Verify by reloading and making a single test prediction
with open(model_filename, 'rb') as file:
    loaded_model = pickle.load(file)
    sample_prediction = loaded_model.predict(X_test[:1])
    print("Verification - Test Prediction for first record:", sample_prediction[0])
    print("Verification - Actual Value for first record:", y_test.iloc[0])

```

Expected Deliverables: As per the guidelines, ensure you save the outputs of these Python scripts in your Jupyter Notebooks (.ipynb) [cite: 1]. You must also zip your notebooks, the /outputs folder from Job 02, and the model.pkl file from Job 03, and submit them to your assessor [cite: 1].